

Assignment 01: Plan It, Then Build It

SDLC • Requirements • Methodology • Git • Deployment

In this assignment, you and a partner will plan a small software product and then demonstrate the Git workflow needed to build it safely as a team. The focus is not on creating a large application. The focus is on showing that you understand how requirements, SDLC decisions, teamwork, version control, and deployment fit together.

Assignment Logistics

Group Size	2 students per group
Total Grade	10 points
Submission	One PDF planning document + one GitHub repository link
Project Choice	Choose a small app idea. Do not use the Pizza App.

Suggested project ideas: Study Group Scheduler, Campus Food Truck Tracker, Habit Tracker, Used Textbook Marketplace, Appointment Reminder, Simple Event Planner, or another small idea approved by the instructor etc.

What You Should Demonstrate

- Apply all six SDLC stages to a specific software idea.
- Turn user needs into clear, testable software requirements.
- Select Waterfall, Agile, or DevOps using project factors rather than personal preference.
- Collaborate with a partner using branches, commits, merging, and conflict resolution in Git.
- Explain how Blue Green or Canary deployment could reduce release risk.

Part 1: The Plan | 5 points

No coding is required in Part 1. Create a concise planning document for your app. Aim for approximately 2 to 3 pages, plus the requirement cards.

1. SDLC Breakdown

For each stage below, write 1 to 2 sentences explaining what that stage would look like for your specific app. Do not simply define the stage.

- Planning
- Requirements Analysis
- Design

- Implementation
- Testing
- Deployment and Maintenance

2. Requirement Cards | Mini SRS Activity

Yes, this is a simplified SRS style activity. Create at least 4 requirement cards for your app. Each card represents one functional requirement. Your cards should be specific enough that another developer could understand what must be built and a tester could decide whether it works.

Requirement writing rule: Write what the system must do, not how you will code it. Use clear statements such as “The system shall allow a student to create a study session.”

Requirement Card Template

Requirement ID	FR 01
Requirement Name	Create Study Session
User / Actor	Student
Requirement Statement	The system shall allow a student to create a study session with a date, time, course, and location.
Priority	Must Have / Should Have / Could Have
Acceptance Criteria	Given valid session details, when the student saves the session, then the new session appears in the session list.
Related SDLC Stage	Requirements Analysis; later verified during Testing

Your 4 cards must include: a unique ID, requirement name, user or actor, “system shall” statement, priority, and at least one acceptance criterion. Keep each card focused on one requirement.

3. Methodology Choice

Answer the three decision factor questions for your project, then choose Waterfall, Agile, or DevOps and justify your choice in one short paragraph.

- Requirement stability: Are requirements fixed or likely to change?
- Client involvement: How often will users or clients provide feedback?
- Release frequency: Is this a one time release or will updates happen often?

4. Team Collaboration Plan

Explain how you and your partner will work together under the methodology you selected. Include each person’s role, how often you will check in, and how you will track progress using a schedule, backlog, task board, dashboard, or another suitable approach.

5. Project Schedule | Gantt or PERT Representation

Create one simple project schedule that shows how your work will be organized over the 10 days. You may use either a Gantt chart or a PERT chart. Your chart must use tasks from your own project and show the order or timing of the work. A Gantt chart example is shown below. Create a similar chart for your project.

Example Gantt Chart: Study Group Scheduler

Project Task	Days 1 to 2	Days 3 to 4	Day 5	Days 6 to 8	Days 9 to 10
Planning and app idea	X				
Requirements and SRS cards		X			
Methodology and team plan			X		
Git branches and project work				X	
Testing, README, and final review					X

Your chart should include at least 5 meaningful project tasks. The schedule should be realistic for the 10 day assignment and should match the work described in your SDLC, requirements, methodology, and Git plan.

6. Deployment Strategy Plan

Choose Blue Green or Canary deployment. Apply the strategy to your app rather than only defining it. Explain what the environments or canary group would look like, how you would verify the new version, and how you would recover if the release caused a problem.

Part 2: The Build | 5 points

You are not required to build the full application. You are demonstrating safe team collaboration with Git. A few simple files are enough as long as the repository shows genuine work from both partners.

1. Create one shared GitHub repository and give both partners push access.
2. Create a README.md that names the project, explains its purpose, and lists the 4 requirement IDs from Part 1.
3. Each partner creates at least one feature branch and makes meaningful changes connected to one or more requirement cards.
4. Each partner makes meaningful, incremental commits. Commit messages must explain what changed. Avoid messages such as “update” or “fix.”
5. Merge both feature branches into main.
6. Deliberately create and resolve at least one merge conflict. Document what caused the conflict and how your team resolved it.
7. Add two simple environment configuration examples, such as .env.development.example and .env.production.example. Use placeholder values only. Do not commit real passwords, API keys, or secrets.
8. In README.md, explain your branching and merging workflow, the merge conflict, and how your selected Blue Green or Canary strategy would apply to the repository.

Important security note: Never place real passwords, tokens, API keys, or other secrets in the repository. Use placeholder values only.

Suggested 10 Day Work Plan

Time	Recommended Work
Days 1 to 2	Choose the app, define its purpose, and complete the SDLC outline.
Days 3 to 4	Write and review the 4 requirement cards. Make sure each requirement is clear and testable.
Day 5	Choose the methodology and create the collaboration and deployment plans.
Days 6 to 8	Create the GitHub repository, branches, commits, environment examples, and merge conflict.
Day 9	Finish README documentation and verify that both students' contributions are visible.
Day 10	Final quality check. Export Part 1 to PDF and submit the PDF plus repository link.

Grading Rubric | 10 points total

Criterion	Pts	What We Are Looking For
SDLC application	0.5	All 6 stages are applied specifically to the chosen app. Responses show application, not copied definitions.
Requirement cards	0.5	At least 4 clear functional requirement cards are included. Each has an ID, actor, system shall statement, priority, and testable acceptance criterion.
Methodology choice and justification	2.0	Waterfall, Agile, or DevOps is selected using all four decision factors with clear project specific reasoning.
Team collaboration plan	0.5	Roles, communication cadence, and progress tracking are clear and consistent with the selected methodology.
Project schedule: Gantt or PERT	0.5	A clear chart includes at least 5 project tasks and realistically represents the order or timing of work across the 10 day assignment.
Deployment strategy plan	1.0	Blue Green or Canary is correctly applied to the app and includes verification and recovery or rollback thinking.

Criterion	Pts	What We Are Looking For
Repository and commit history	2.0	Both partners contribute meaningful incremental commits. Commit history clearly shows shared work and useful messages.
Branching, merging, and conflict resolution	2.0	Each partner uses at least one feature branch. Branches are merged to main and at least one deliberate conflict is correctly resolved and documented.
README and environment configuration	1.0	README documents the workflow, conflict resolution, requirement IDs, and deployment strategy. Separate safe development and production example configuration files are present.

What to Submit

- One PDF containing Part 1: SDLC breakdown, 4 requirement cards, methodology choice, collaboration plan, one Gantt or PERT chart, and deployment plan.
- One GitHub repository link for Part 2. Both partners must have visible commits in the repository history.
- Both students' full names and GitHub usernames must appear in the planning document or README.
- Both student submits for the group and are responsible for checking that the submitted links work.

Before You Submit: Quick Check

- ☐ We included all 6 SDLC stages.
 - ☐ We created at least 4 requirement cards with testable acceptance criteria.
 - ☐ We justified our methodology using all 4 decision factors.
 - ☐ We created a Gantt or PERT chart with at least 5 project tasks.
-
- ☐ Both partners made meaningful commits.
 - ☐ Both partners used a feature branch.
 - ☐ We created and resolved a merge conflict.
 - ☐ Our README explains our Git workflow and deployment strategy.
 - ☐ We did not commit any real passwords, API keys, or secrets.